

LIM M2

Архітектурний довідник процесора

Високодетермінована 16-розрядна мікропрограмна архітектура з чистим мультиплексорним трактом даних та розширеною 24-бітною адресацією.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

OFFICIAL REFERENCE SPECIFICATION

Редакція 0.2 — Жовтень 2026 р.

Специфікація процесора LIM M2. Всі права захищено.

Contents

1 Статус документа та архітектурна концепція	3
1.1 Умовні позначення	3
1.2 Єдина філософія формату інструкцій	3
2 Архітектурний огляд процесора LIM M2	3
2.1 Концептуальна схема тракту даних	5
3 Програмно доступні регістри та стан процесора	6
3.1 Карта регістра прапорців та керування (FREG)	6
4 Формат машинних команд	7
5 Зведена таблиця команд ядра LIM M2	7
6 Детальна специфікація системи команд	8
NOP — Апаратна порожня операція та скидання ланцюжка обчислень	9
ADD — Додавання з автоматичним каскадуванням переносу	11
SUB — Віднімання з автоматичним каскадуванням позики	13
MUL — Цілочисельне множення зі збереженням старшої частини в регістрі АЛП	15
DIV — Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП	17
LMR — Завантаження даних із пам'яті та зовнішніх пристроїв у регістри	19
LRM — Запис даних із регістрів у пам'ять та зовнішні пристрої	21
LRR — Міжрегістрове переміщення (копіювання з точки Б в точку А)	23
CMP — Апаратне порівняння операндів	25
JMP — Безумовний перехід	27
JCR — Умовний перехід за прапорцем переносу	29
JNZ — Умовний перехід, якщо не нуль	31
JZ — Умовний перехід за нулем	33
AND — Побітове логічне І	35
OR — Побітове логічне АБО	37
NAND — Побітове логічне І-НІ	39
NOR — Побітове логічне АБО-НІ	41
NOT — Побітова інверсія	43
XOR — Побітове виключне АБО	45
XNOR — Побітова еквівалентність	47
INC — Інкремент на одиницю	49
DEC — Декремент на одиницю	51
PUSH — Збереження слова в стек	53
POP — Вилучення слова з вершини стека	55

BSL — Барельний зсув ліворуч	57
BSR — Барельний зсув праворуч	59
MSP — Маніпуляція спеціальними регістрами (FREG, PTREG)	61
RSV27 — Апаратне розширення	63
RSV28 — Апаратне розширення	64
RSV29 — Апаратне розширення	65
RSV30 — Апаратне розширення	66
RSV31 — Апаратне розширення	67
7 Фізичний інтерфейс та цоколівка (34-вивідний корпус)	68
7.1 Опис системних сигналів	68
8 Підсистема переривань та векторна обробка	69
8.1 Єдиний простір 16 векторів переривань	69
8.2 Апаратний протокол квітування (RDI → IA → D0–D7)	69
8.3 Збереження контексту у 24-бітному стеку та вхід в обробник	70
8.4 Повернення з переривання	70
9 Висновки	71

1 Статус документа та архітектурна концепція

Цей документ є офіційним технічним описом та архітектурним стандартом 16-розрядного мікропроцесора **LIM M2**. Архітектура спроектована з фокусом на абсолютну передбачуваність, математичну точність і високу тактову частоту.

Ключовим рішенням у тракті даних LIM M2 стала **повна відмова від внутрішніх шин третього стану (Tri-State / Hi-Z)**. Внутрішні потоки даних побудовані на основі активних мультиплексорних дерев (Point-to-Point MUX Routing), що цілком усуває ризик конфліктів шини (bus contention), наскрізних струмів та паразитних ємнісних затримок.

Нумерація системи команд

Система команд LIM M2 використовує суцільний 5-бітний діапазон базових опкодів: NOP=0, ADD=1, ..., MSP=26. Опкоди 27–31 зарезервовано для розширень наступного покоління.

1.1 Умовні позначення

Позначення	Архітектурне значення
dst	Цільовий регістр-приймач результату операції
src, srcA, srcB	Регістри-джерела вхідних операндів
imm	Безпосереднє скалярне значення в тілі команди
addr	Ефективна адреса пам'яті (16-бітна базова або 24-бітна розширена)
cfg	3-бітне поле конфігураційного модифікатора операції
ZF, CF, OF	Апаратні прапорці нуля, переносу та переповнення регістра FREG

1.2 Єдина філософія формату інструкцій

Архітектура процесора LIM M2 суворо дотримується універсальної парадигми: **спочатку завжди вказується цільовий приймач (куди), а потім вхідні операнди (звідки)**. Це забезпечує стрункий, логічний та інтуїтивно зрозумілий синтаксис:

- **Арифметичні операції (3 операнди):** ADD Rd, Rs1, Rs2 означає $Rd \leftarrow Rs1 + Rs2$ (наприклад, ADD R0, R1, R2 обчислює $R0 = R1 + R2$); аналогічно для SUB, MUL, DIV.
- **Логічні операції (2 операнди):** AND Rd, Rs означає $Rd \leftarrow Rd \text{ AND } Rs$ (наприклад, AND R0, R1 обчислює $R0 = R0 \& R1$); аналогічно для OR, NAND, NOR, XOR, XNOR.
- **Побітова інверсія NOT (1 операнд, 1-байтова інструкція):** NOT Rd означає $Rd \leftarrow \sim Rd$ (наприклад, NOT R0 обчислює $R0 = \sim R0$). Кодується в 1 байті ([7:3] опкод, [2:0] регістр Rd), другий байт із пам'яті не запитується.
- **Міжрегістрові пересилання:** LRR Rd, Rs означає пряме копіювання з точки Б (джерело Rs) у точку А (приймач Rd): $Rd \leftarrow Rs$ (наприклад, LRR R0, R1 означає $R0 = R1$).
- **Завантаження з пам'яті (LMR):** перший аргумент Rd визначає цільовий регістр процесора, куди записуються зчитані дані.

2 Архітектурний огляд процесора LIM M2

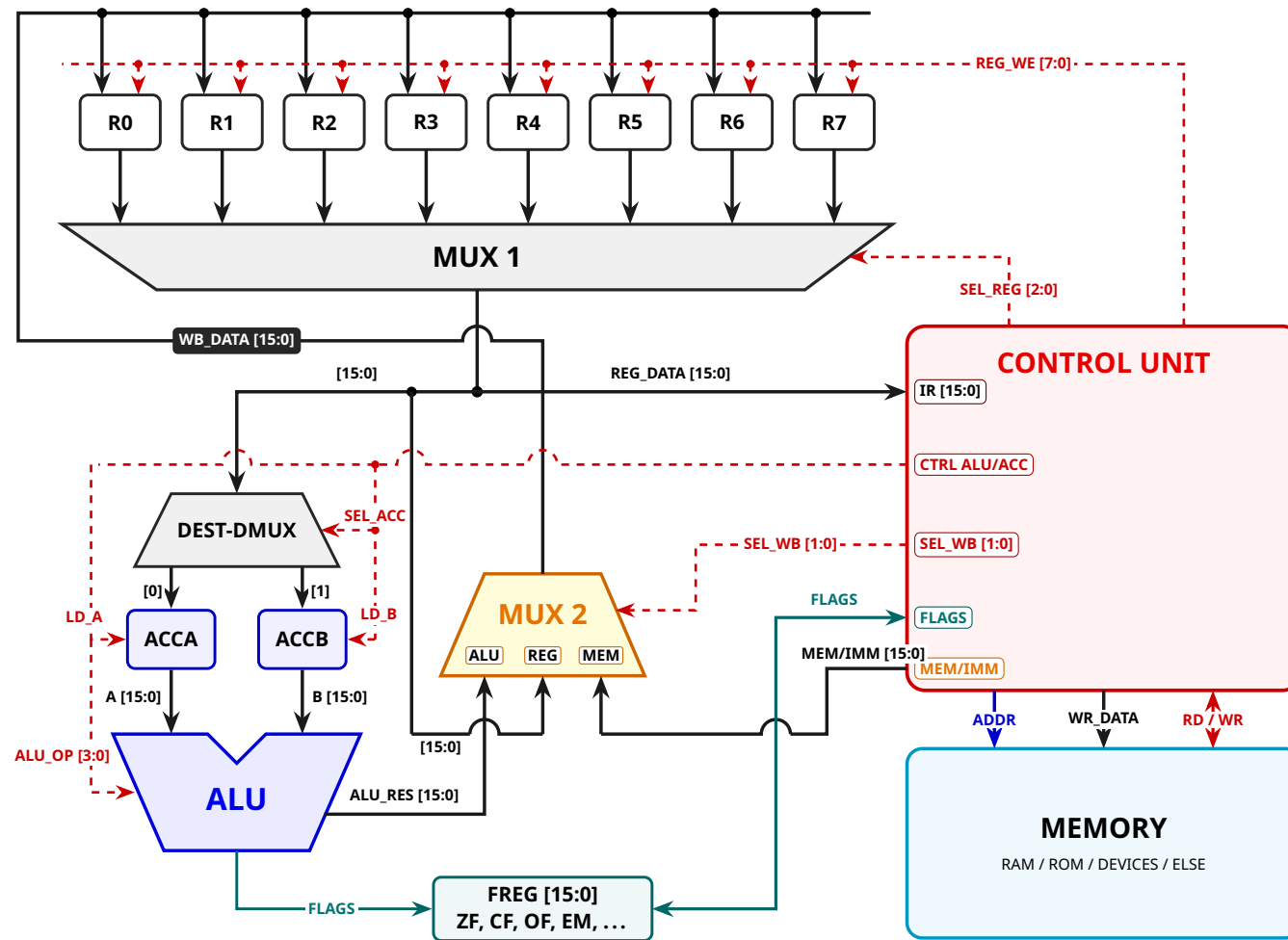
LIM M2 — високопродуктивний 16-розрядний процесор із мікропрограмним керуванням, орієнтований на цілочисельні обчислення. Основні архітектурні переваги:

- **Чиста мультиплексорна маршрутизація:** Тракт даних використовує виключно активні

драйвери через каскад селекторів **MUX 1**, **DEST-DMUX** та **MUX 2**. Це гарантує ідеальну сумісність із кремнієвими технологіями (ASIC) та сучасними ПЛІС (FPGA).

- **Двоакумуляторний виконавчий вузол:** Регістри ACCA та ACCB надійно фіксують операнди перед АЛП, ізолюючи перехідні процеси від регістрового файлу.
- **Апаратне каскадування довгої арифметики (ALU Auto-Chaining):** Внутрішній тракт АЛП містить залишковий акумулятор (ALU_RESIDUAL). У разі переносу ($+1$ при сумі $> 2^{16} - 1$), позики (-1 при різниці < 0) або надлишкових старших розрядів множення/ділення, залишок фіксується апаратно та автоматично додається чи віднімається під час наступної математичної операції. Це забезпечує наскрізну багатобайтову арифметику (32, 48, 64+ бітів) без окремих команд ADC/SBB. Скидання ланцюга виконується командою NOP.
- **Прямий канал пересилання (Reg-to-Reg Bypass):** Обхідний шлях із MUX 1 безпосередньо в MUX 2 дозволяє копіювати дані між регістрами за мінімальний час, не блокуючи АЛП.
- **Лінійна 24-бітна адресація пам'яті (до 16 МБ):** Спеціалізовані 24-бітні регістри PTREG (показчик даних з автоінкрементом) та SREG (показчик стека) дозволяють лінійно адресувати пам'ять та стек до 16 Мегабайтів.
- **Високий потенціал тактової частоти:** Завдяки відсутності ємнісного навантаження розподілених шин ядро розраховане на базову частоту **200 МГц**, а в оптимізованому кремнії досягає **700 МГц і вище**.

2.1 Концептуальна схема тракту даних



3 Програмно доступні регістри та стан процесора

Архітектурний стан процесора LIM M2 утворює чітку ортогональну систему регістрів:

Регістр	Розрядність	Призначення та ключові особливості
R0–R7	16 бітів	Регістри загального призначення (GPR). Селектор MUX 1 забезпечує швидке зчитування; запис відбувається за індивідуальними стробами з шини WB_DATA.
SREG	24 біти	Системний регістр стека (Stack Register). Забезпечує апаратну підтримку викликів процедур і кадрів стека з прямою 24-бітною адресацією стекового простору (до 16 Мбайт).
PTREG	24 біти	Високоточний регістр-показчик (Pointer Register). Забезпечує суцільну адресацію пам'яті до 16 Мбайт з апаратним автоінкрементом за прапорцем AIP.
FREG	16 бітів	Регістр стану та керування (Processor Status & Control Register). Об'єднує арифметичні прапорці результату та біти конфігурації ядра.

3.1 Карта регістра прапорців та керування (FREG)

Регістр FREG координує функціонування обчислювального ядра:

Біт	Мнемоніка	Функціональний опис
0	ZF	Zero Flag: істинний при нульовому результаті операції.
1	CF	Carry Flag: апаратний перенос або позика у знаковому/беззнаковому режимах.
2	OF	Overflow Flag: індикатор знакового переповнення додаткового коду.
3	EM	Error Math: апаратний прапорець математичних помилок (ділення на нуль).
4–5	CMP	Compare Result: результат апаратного компаратора (00: скидання/не порівнювалось/помилка, 01: менше, 10: дорівнює, 11: більше).
6–7	CLK	Clock Mode: конфігурація тактового генератора та дільників частоти.
8	PIE	Program Interrupt Enable: глобальний дозвіл програмних переривань.
9	MIE	Master Interrupt Enable: дозвіл зовнішніх апаратних переривань.
10	DWR	Disable Wait for Ready: форсований обмін без очікування сигналу готовності шини.
11	M24	24-bit Mode Enable: активація розширеного 24-бітного лінійного адресного простору.
12	AIP	Auto-Increment PTREG: дозвіл апаратного автоінкременту показчика PTREG.
13	DBIT	Displaced Bit: буфер витісненого біта зсувного вузла.
14	SMOD	Shift Mode: керування циклічним режимом зсуву (0: заповнення нулями, 1: прохід через DBIT).
15	INTR	In Interrupt: прапорець перебування у перериванні (при 1 нові переривання апаратно заблоковані).

4 Формат машинних команд

Команди процесора LIM M2 оптимізовані для швидкого декодування. Старші 5 бітів відведено під базовий опкод:

$W - 1$	$W - 6$	$W - 9$	0
OPCODE 5 бітів	CFG 3 біти	Операнди та аргументи динамічна довжина	

Поле cfg (3 біти) визначає до 8 незалежних модифікацій базової інструкції, забезпечуючи високу виразність команд без перевитрати простору опкодів.

5 Зведена таблиця команд ядра LIM M2

№	Код	Мнемоніка	Клас	Функціональне призначення
0	00000	NOP	Керування	Порожня операція та скидання залишкового регістра АЛП
1	00001	ADD	Арифметика	Додавання з автокаскадуванням переносу через залишок АЛП
2	00010	SUB	Арифметика	Віднімання з автокаскадуванням позики через залишок АЛП
3	00011	MUL	Арифметика	Множення з фіксацією старших розрядів у залишку АЛП
4	00100	DIV	Арифметика	Ділення з фіксацією залишку у внутрішньому регістрі АЛП
5	00101	LMR	Пам'ять	Завантаження з пам'яті/вводу в регістр (конфіг джерела 0-3, розмір 8/16 біт)
6	00110	LRM	Пам'ять	Запис даних з регістра в пам'ять/вивід (конфіг призначення 0-3, розмір 8/16 біт)
7	00111	LRR	Пересилання	Пряме міжрегістрове копіювання через обхідний MUX
8	01000	CMP	Порівняння	Апаратне порівняння без спотворення операндів
9	01001	JMP	Перехід	Безумовний перехід (прямий, непрямий, 24-бітний)
10	01010	JCR	Перехід	Умовний перехід за прапорцем переносу CF
11	01011	JNZ	Перехід	Умовний перехід, якщо результат не нуль (ZF = 0)
12	01100	JZ	Перехід	Умовний перехід за нульовим результатом (ZF = 1)
13	01101	AND	Логіка	Побітове логічне множення (I)
14	01110	OR	Логіка	Побітове логічне додавання (АБО)
15	01111	NAND	Логіка	Побітове логічне І-НІ
16	10000	NOR	Логіка	Побітове логічне АБО-НІ
17	10001	NOT	Логіка	Побітова інверсія операнда (доповнення до 1)
18	10010	XOR	Логіка	Побітове виключне АБО
19	10011	XNOR	Логіка	Побітова еквівалентність
20	10100	INC	Арифметика	Апаратний інкремент значення на 1
21	10101	DEC	Арифметика	Апаратний декремент значення на 1
22	10110	PUSH	Стек	Розміщення слова у стековому просторі
23	10111	POP	Стек	Вилучення вершини стека в цільовий регістр

№	Код	Мнемоніка	Клас	Функціональне призначення
24	11000	BSL	Зсуви	Барельний зсув ліворуч із налаштуванням бітом
25	11001	BSR	Зсуви	Барельний зсув праворуч із фіксацією в DBIT
26	11010	MSP	Система	Маніпуляція службовими регістрами (FREG, PTREG)
27	11011	RSV27	Резерв	Зарезервовано для векторних інструкцій
28	11100	RSV28	Резерв	Зарезервовано для розширеної адресації
29	11101	RSV29	Резерв	Зарезервовано для операцій із рухомою комою
30	11110	RSV30	Резерв	Зарезервовано для апаратної криптографії
31	11111	RSV31	Резерв	Зарезервовано для майбутніх розширень

6 Детальна специфікація системи команд

Нижче подано повний архітектурний опис усіх 32 опкодів ядра LIM M2.

NOP

00000

Керування

Апаратна порожня операція та скидання ланцюжка обчислень

Мнемоніка	NOP
Opcode	00000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Керування
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратна порожня операція та скидання ланцюжка обчислень.

Архітектурна семантика

Сигнатура: NOP

Операція: $PC \leftarrow PC + 1$, $ALU_RESIDUAL \leftarrow 0$, $FREG.CF \leftarrow 0$, $FREG.OF \leftarrow 0$

Приклад: NOP ; Скидання ланцюжка переносу АЛП

Конфігурація та операнди

Операнди відсутні; поле cfg зарезервовано для калібрування затримок.

Вплив на регістр прапорців (FREG)

Скидає прапорці переносу (CF), переповнення (OF) та позики; інші прапорці зберігаються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Виконується для тактової синхронізації конвеєра та апаратного скидання (Flush) внутрішнього залишкового регістра АЛП, розриваючи ланцюг автопереносу перед новими незалежними обчисленнями.

Поля з позначкою TBD визначають розширені параметри специфікації.

ADD

00001

Арифметика

Додавання з автоматичним каскадуванням переносу

Мнемоніка	ADD
Opcode	00001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Додавання з автоматичним каскадуванням переносу.

Архітектурна семантика

Сигнатура: `ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2`
 Операція: `sum <- arg1 + arg2 + ALU_RESIDUAL`
`arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 : 0)`
 Приклад: `ADD R0, R1, R2 ; R0 = R1 + R2 (регістри R0–R7)`

Конфігурація та операнди

Регістри R0–R7, акумулятори ACCA/ACCB або безпосередні операнди.

Вплив на регістр прапорців (FREG)

Встановлює прапорець переносу CF та переповнення OF при перевищенні $2^{16} - 1$; оновлює ZF за значенням dst.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Забезпечує наскрізну багатобайтову арифметику (32, 48, 64+ бітів). Якщо результат суми перевищує 16 бітів ($> 2^{16} - 1$), перенос (+1) фіксується у внутрішньому регістрі АЛП та автоматично додається під час наступної операції ADD без потреби в окремих інструкціях ADC. Молодші 16 бітів детерміновано зберігаються в dst. Ланцюг скидається командою NOP або перезаписується новою математичною операцією.

Поля з позначкою TBD визначають розширені параметри специфікації.

SUB

00010

Арифметика

Віднімання з автоматичним каскадуванням позики

Мнемоніка	SUB
Opcode	00010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Віднімання з автоматичним каскадуванням позики.

Архітектурна семантика

Сигнатура: SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2
 Операція: diff <- arg1 - arg2 - ALU_RESIDUAL
 arg0 <- diff[15:0], ALU_RESIDUAL <- (diff < 0 ? 1 : 0)
 Приклад: SUB R0, R1, R2 ; R0 = R1 - R2 (регістри R0–R7)

Конфігурація та операнди

Регістри R0–R7, константи або акумулятори ACCA/ACCB.

Вплив на регістр прапорців (FREG)

При різниці < 0 встановлює прапорець позики (Borrow / CF) та OF; оновлює ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Реалізує неперервне багатослівне віднімання. При виникненні від'ємної різниці (< 0) позика (-1) зберігається у внутрішньому регістрі АЛП, і в наступній операції SUB АЛП автоматично віднімає 1 зі старшого слова. Результат молодшого слова детерміновано фіксується в `dst`. Для початку нової незалежної різниці ланцюг переривається інструкцією NOP.

Поля з позначкою TBD визначають розширені параметри специфікації.

MUL

00011

Арифметика

Цілочисельне множення зі збереженням старшої частини в регістрі АЛП

Мнемоніка	MUL
Opcode	00011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Цілочисельне множення зі збереженням старшої частини в регістрі АЛП.

Архітектурна семантика

```
Сигнатура: MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2
Операція: prod <- arg1 * arg2
arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]
Приклад: MUL R0, R1, R2 ; R0 = R1 * R2 (старша частина в ALU_RESIDUAL)
```

Конфігурація та операнди

Регістри R0–R7, акумулятори ACCA/ACCB.

Вплив на регістр прапорців (FREG)

Оновлює ZF; встановлює OF/CF, якщо старші розряди добутку (prod[31:16]) не дорівнюють 0.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

При множенні молодші 16 бітів добутку надходять у цільовий регістр `dst`, а старші 16 бітів фіксуються у внутрішньому службовому регістрі АЛП (ALU Residual). У наступній операції додавання цей залишок автоматично підсумовується (апаратна підтримка MAC / довгих чисел). Старшу частину можна також отримати в регістр через додавання очищеного регістра з самим собою. Команда `NOP` очищує залишковий регістр АЛП.

Поля з позначкою TBD визначають розширені параметри специфікації.

DIV

00100

Арифметика

Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП

Мнемоніка	DIV
Opcode	00100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП.

Архітектурна семантика

Сигнатура: DIV arg0(REG), arg1(REG), arg2(REG) : $\text{arg0} = \text{arg1} / \text{arg2}$
 Операція: $\text{arg0} \leftarrow \text{arg1} / \text{arg2}$
 $\text{ALU_RESIDUAL} \leftarrow \text{arg1} \% \text{arg2}$
 Приклад: DIV R0, R1, R2 ; $R0 = R1 / R2$ (остача modulo в ALU_RESIDUAL)

Конфігурація та операнди

Регістри R0–R7, акумулятори ACCA/ACCB.

Вплив на регістр прапорців (FREG)

При діленні на нуль зводиться прапорець помилки EM (Error Math) без блокування ядра; оновлює ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Результат цілочисельного ділення (частка) записується в цільовий регістр `dst`, а залишок від ділення (`modulo`) автоматично зберігається у внутрішньому службовому регістрі АЛП. Збережений залишок може бути задіяний у подальших операціях АЛП або скинутий командою `NOP`.

LMR

00101

Пам'ять

Завантаження даних із пам'яті та зовнішніх пристроїв у регістри

Мнемоніка	LMR
Opcode	00101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Пам'ять
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Завантаження даних із пам'яті та зовнішніх пристроїв у регістри.

Архітектурна семантика

Сигнатура: LMR arg0(REG), <SRC_MODE> : arg0 = MEM[...]
 Режими: SRC=0: LMR R0, [PC+] ; Imm Data (1-2 байти)
 SRC=1: LMR R0, [[PC+]] ; Imm Pointer
 SRC=2: LMR R0, [PTREG] ; Системний покажчик (AIP)
 SRC=3: LMR R0(R_hi), R1(R_lo) ; 24-біт адреса з пари
 Операція: arg0 <- MEM[EA] (старші біти адреси перезаписуються)
 Приклад: LMR R0, R1 ; EA=(R0«16)|R1, читання MEM[EA] в R0

Конфігурація та операнди

Октет 0: біт [8] (SZ) задає розрядність (0 = 1 байт, 1 = 2 байти / слово); бити [10 : 9] (SRC) задають джерело даних (0 — дані безпосередньо за кодом команди, 1 — безпосередній покажчик за командою, 2 — непрямо через 24-бітний покажчик PTREG, 3 — непрямо через пару регістрів користувача).

Октет 1: біти [7 : 5] задають регістр Arg1 (R_hi / приймач), біти [4 : 2] задають регістр Arg2 (R_lo). У режимі SRC = 3 два 16-бітних регістри утворюють 32-бітне число, з якого для фізичної адресації використовуються 24 біти: Arg1 містить старші біти адреси, а Arg2 — молодші 16 біт. При цьому Arg1 одночасно виступає цільовим регістром: після вибірки з пам'яті старші біти адреси в ньому перезаписуються зчитаними даними.

Вплив на регістр прапорців (FREG)

Усі прапорці регістра стану FREG зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Інструкція LMR є фундаментальним шлюзом введення зовнішньої інформації в регістровий файл процесора LIM M2 ($R0-R7$). Архітектура підтримує чотири ортогональні режими вибірки та повну 24-бітну адресацію:

1. У режимі $SRC = 0$ процесор зчитує константу безпосередньо з потоку коду (1 або 2 байти) із просуванням лічильника команд PC.
2. У режимі $SRC = 1$ за інструкцією зчитується абсолютний показник, за яким завантажуються дані.
3. У режимі $SRC = 2$ читання виконується за 24-бітною фізичною адресою із системного регістра PTREG. За активного прапорця автоінкременту FREG.AIP (біт 12) регістр PTREG автоматично інкрементується на розмір вибірки (+1 або +2), забезпечуючи потокове DMA-подібне введення масивів за один такт.
4. У режимі $SRC = 3$ користувач вказує два 16-бітних регістри (Arg1 та Arg2), які разом утворюють 32-бітне число, з якого на практиці адресації використовуються молодші 24 біти. Користувач визначає, що старші біти адреси знаходяться в Arg1, а молодші 16 біт адреси — в Arg2. У цьому режимі регістр Arg1 зі старшими бітами адреси також стає регістром призначення, куди зберігаються зчитані дані (тобто старші біти адреси в Arg1 перезаписуються даними, тоді як Arg2 зберігає молодші 16 біт адреси).

Поля з позначкою TBD визначають розширені параметри специфікації.

LRM

00110

Пам'ять

Запис даних із регістрів у пам'ять та зовнішні пристрої

Мнемоніка	LRM
Opcode	00110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Пам'ять
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Запис даних із регістрів у пам'ять та зовнішні пристрої.

Архітектурна семантика

Сигнатура: LRM arg0(REG), <DST_MODE> : MEM[...] = arg0
 Режими: DST=0: LRM R0, [PC+] ; Вбудована пам'ять за кодом
 DST=1: LRM R0, [[PC+]] ; Imm Pointer
 DST=2: LRM R0, [PTREG] ; Системний покажчик (AIP)
 DST=3: LRM R0(R_hi), R1(R_lo) ; 24-біт адреса з пари
 Операція: MEM[EA] <- arg0[15:0] (або молодший байт при SZ=0)
 Приклад: LRM R0, R1 ; EA=(R0«16)|R1, запис R0 в MEM[EA]

Конфігурація та операнди

Октет 0: біт [8] (SZ) задає розрядність (0 = 1 байт, 1 = 2 байти / слово); бити [10 : 9] (DST) задають цільовий приймач даних (0 — запис безпосередньо за кодом команди, 1 — запис за покажчиком за командою, 2 — непрямо через 24-бітний покажчик PTREG, 3 — непрямо через пару регістрів користувача).

Октет 1: біти [7 : 5] задають регістр-джерело Arg1 (Rs / R_hi), біти [4 : 2] задають регістр Arg2 (R_lo). У режимі DST = 3 два регістри формують 32-бітне число, з якого використовуються 24 біти адреси (Arg1 — старші біти адреси та джерело даних, Arg2 — молодші 16 біт). Знищення наявних даних за адресою запису є відповідальністю програміста.

Вплив на регістр прапорців (FREG)

Усі прапорці регістра стану FREG зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Інструкція LRM є фундаментальним симетричним механізмом вивантаження регістрових даних процесора LIM M2 (*R0–R7*) в оперативну пам'ять та зовнішні порти периферії. Архітектура підтримує чотири ортогональні режими адресації:

1. У режимі $DST = 0$ процесор записує 1 або 2 байти безпосередньо в пам'ять слідом за інструкцією (PC) зі зсувом лічильника команд. Режим застосовний для динамічних буферів та саомодифікації коду; перезапис даних лежить на відповідальності програміста.
2. У режимі $DST = 1$ за інструкцією зчитується 16-бітний абсолютний адресний показник, за яким зберігаються дані.
3. У режимі $DST = 2$ запис виконується за 24-бітною системною адресою регістра PTREG. За активного прапорця FREG.AIP (біт 12) регістр PTREG автоматично інкрементується на +1 або +2, забезпечуючи апаратне потокове DMA-виведення масивів у пам'ять без програмних циклів.
4. У режимі $DST = 3$ використовується пара 16-бітних регістрів: Arg1 та Arg2 утворюють 32-бітне складене число, з якого молодші 24 біти задають фізичну адресу (Addr[23 : 0]). Дані з Arg1 записуються за обчисленою адресою, забезпечуючи прямий доступ до всього адресного простору 16 МБ.

Поля з позначкою TBD визначають розширені параметри специфікації.

LRR

00111

Пересилання

Міжрегістрове переміщення (копіювання з точки Б в точку А)

Мнемоніка	LRR
Opcode	00111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Пересилання
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Міжрегістрове переміщення (копіювання з точки Б в точку А).

Архітектурна семантика

Сигнатура: LRR arg0(REG), arg1(REG) : arg0 = arg1
 Операція: arg0 <- arg1 (пряме копіювання з точки Б в точку А)
 Приклад: LRR R0, R1 ; R0 = R1 (значення R1 копіюється в R0)

Конфігурація та операнди

Октет 0: код операції 00111₂ (0x07), біти [10 : 8] зарезервовані (000₂).

Октет 1: біти [7 : 5] задають цільовий регістр-приймач Arg1 (Rd, точка А, куди зберігаються дані); біти [4 : 2] задають регістр-джерело Arg2 (Rs, точка Б, звідки беруться дані); біти [1 : 0] містять 00₂. При виконанні значення Rs копіюється в Rd за 1 такт.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Інструкція LRR є фундаментальною операцією пересилання даних між регістрами загального призначення процесора ($R0-R7$). Вона здійснює передачу з джерела (точка Б) у приймач (точка А). Пересилання виконується через прямий апаратний обхідний тракт **MUX 1** → **MUX 2** в обхід АЛП, що виключає блокування обчислювального ядра та не змінює стан прапорців FREG.

Поля з позначкою TBD визначають розширені параметри специфікації.

CMP

Апаратне порівняння операндів

01000

Порівняння

Мнемоніка	CMP
Opcode	01000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Порівняння
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратне порівняння операндів.

Архітектурна семантика

Сигнатура: `CMP arg0(REG), arg1(REG) : compare(arg0, arg1)`
 Операція: `if arg0 < arg1 then FREG.CMP <- 01b (менше)`
`if arg0 == arg1 then FREG.CMP <- 10b (дорівнює)`
`if arg0 > arg1 then FREG.CMP <- 11b (більше)`
 (стан 00b — початкове скидання / не порівнювалось / помилка)
 Приклад: `CMP R0, R1 ; 01: R0<R1, 10: R0==R1, 11: R0>R1 (00: скидання/помилка)`

Конфігурація та операнди

Порівняння регістрів без зміни даних.

Вплив на регістр прапорців (FREG)

Оновлює 2-бітне поле FREG.CMP (біти [5:4]), а також ZF, CF, OF без зміни регістрів.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Забезпечує швидке розгалуження алгоритмів.

JMP

Безумовний перехід

01001

Перехід

Мнемоніка	JMP
Opcode	01001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Безумовний перехід.

Архітектурна семантика

Сигнатура: `JMP target(IMM|REG) : PC = target`
Операція: `PC <- target`
Приклад: `JMP R0 ; Безумовний перехід за адресою в R0`

Конфігурація та операнди

Підтримує 16-бітний локальний або повний 24-бітний перехід через PTREG.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Атомарне перезавантаження Program Counter за мінімум тактів.

Поля з позначкою TBD визначають розширені параметри специфікації.

JCR

Умовний перехід за прапорцем переносу

01010

Перехід

Мнемоніка	JCR
Opcode	01010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Умовний перехід за прапорцем переносу.

Архітектурна семантика

Сигнатура: JCR target(IMM|REG) : if FREG.CF == 1 then PC = target
 Операція: if FREG.CF == 1 then PC <- target
 Приклад: JCR R2 ; Перехід за адресою R2 при наявності переносу

Конфігурація та операнди

Цільова адреса задається зміщенням або абсолютним значенням.

Вплив на регістр прапорців (FREG)

Прапорці читаються без модифікації.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Фундаментальний примітив для реалізації довгої арифметики.

Поля з позначкою TBD визначають розширені параметри специфікації.

JNZ

Умовний перехід, якщо не нуль

01011

Перехід

Мнемоніка	JNZ
Opcode	01011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Умовний перехід, якщо не нуль.

Архітектурна семантика

Сигнатура: JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target
Операція: if FREG.ZF == 0 then PC <- target
Приклад: JNZ R3 ; Перехід за адресою R3 при ZF == 0 (не нуль)

Конфігурація та операнди

Високоєфективний перехід для організації циклів.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Мінімізований час ухвалення рішення в мікрокоді.

Поля з позначкою TBD визначають розширені параметри специфікації.

JZ

Умовний перехід за нулем

01100

Перехід

Мнемоніка	JZ
Opcode	01100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Умовний перехід за нулем.

Архітектурна семантика

Сигнатура: JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target
Операція: if FREG.ZF == 1 then PC <- target
Приклад: JZ R4 ; Перехід за адресою R4 при ZF == 1 (нуль)

Конфігурація та операнди

Оцінює успішність перевірки або збігу.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Оптимізовано для алгоритмів пошуку.

Поля з позначкою TBD визначають розширені параметри специфікації.

AND

Побітове логічне І

01101

Логіка

Мнемоніка	AND
Opcode	01101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове логічне І.

Архітектурна семантика

Сигнатура: `AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1`
Операція: `arg0 <- arg0 AND arg1`
Приклад: `AND R0, R1 ; R0 = R0 & R1`

Конфігурація та операнди

Регістрові операнди (2 операнди).

Вплив на регістр прапорців (FREG)

Оновлює прапорець нуля ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: [TBD](#).

Особливості реалізації та примітки

Базова операція для накладання бітових масок.

Поля з позначкою TBD визначають розширені параметри специфікації.

OR

Побітове логічне АБО

01110

Логіка

Мнемоніка	OR
Opcode	01110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове логічне АБО.

Архітектурна семантика

Сигнатура: OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1
Операція: arg0 <- arg0 OR arg1
Приклад: OR R0, R1 ; R0 = R0 | R1

Конфігурація та операнди

Регістрові операнди (2 операнди).

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Швидке об'єднання бітів керування.

Поля з позначкою TBD визначають розширені параметри специфікації.

NAND

Побітове логічне I-HI

01111

Логіка

Мнемоніка	NAND
Opcode	01111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове логічне I-HI.

Архітектурна семантика

Сигнатура: `NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)`
Операція: `arg0 <- NOT (arg0 AND arg1)`
Приклад: `NAND R0, R1 ; R0 = ~(R0 & R1)`

Конфігурація та операнди

Універсальний логічний базис (2 операнди).

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Дозволяє синтезувати компактний двійковий код.

Поля з позначкою TBD визначають розширені параметри специфікації.

NOR

Побітове логічне АБО-НІ

10000

Логіка

Мнемоніка	NOR
Opcode	10000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове логічне АБО-НІ.

Архітектурна семантика

Сигнатура: `NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)`
Операція: `arg0 <- NOT (arg0 OR arg1)`
Приклад: `NOR R0, R1 ; R0 = ~(R0 | R1)`

Конфігурація та операнди

Повноцінна інверсна логіка (2 операнди).

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Розширює набір швидких перетворень.

Поля з позначкою TBD визначають розширені параметри специфікації.

NOT

Побітова інверсія

10001

Логіка

Мнемоніка	NOT
Opcode	10001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітова інверсія.

Архітектурна семантика

Сигнатура: NOT arg0(REG) : arg0 = ~arg0
Операція: arg0 <- NOT arg0
Приклад: NOT R0 ; R0 = ~R0 (1-байтова інструкція)

Конфігурація та операнди

Однобайтова інструкція ([7:3] опкод 10001, [2:0] Rd). Другий байт із пам'яті не запитується.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Виконується за мінімальну кількість мікротактів без вибірки другого байта.

Поля з позначкою TBD визначають розширені параметри специфікації.

XOR

Побітове виключне АБО

10010

Логіка

Мнемоніка	XOR
Opcode	10010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове виключне АБО.

Архітектурна семантика

Сигнатура: XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1

Операція: arg0 <- arg0 XOR arg1

Приклад: XOR R0, R1 ; R0 = R0 ^ R1 (при R0==R1 обнуляє R0)

Конфігурація та операнди

Регістрові операнди (2 операнди).

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Основа для алгоритмів контрольних сум та шифрування.

Поля з позначкою TBD визначають розширені параметри специфікації.

XNOR

Побітова еквівалентність

10011

Логіка

Мнемоніка	XNOR
Opcode	10011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітова еквівалентність.

Архітектурна семантика

Сигнатура: $\text{XNOR } \text{arg0}(\text{REG}), \text{arg1}(\text{REG}) : \text{arg0} = \sim(\text{arg0} \wedge \text{arg1})$
Операція: $\text{arg0} \leftarrow \text{NOT } (\text{arg0} \text{ XOR } \text{arg1})$
Приклад: $\text{XNOR } \text{R0}, \text{R1} ; \text{R0} = \sim(\text{R0} \wedge \text{R1})$

Конфігурація та операнди

Побітове порівняння на тотожність (2 операнди).

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Швидка перевірка відповідності бітових масок.

Поля з позначкою TBD визначають розширені параметри специфікації.

INC

Інкремент на одиницю

10100

Арифметика

Мнемоніка	INC
Opcode	10100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Інкремент на одиницю.

Архітектурна семантика

Сигнатура: `INC arg0(REG) : arg0 = arg0 + 1`

Операція: `arg0 <- arg0 + 1`

Приклад: `INC R0 ; R0 = R0 + 1` (інкремент регістра)

Конфігурація та операнди

Швидкий крок лічильника.

Вплив на регістр прапорців (FREG)

Оновлює ZF, OF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Оптимізує обробку масивів даних.

Поля з позначкою TBD визначають розширені параметри специфікації.

DEC

Декремент на одиницю

10101

Арифметика

Мнемоніка	DEC
Opcode	10101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Декремент на одиницю.

Архітектурна семантика

Сигнатура: $\text{DEC } \text{arg0}(\text{REG}) : \text{arg0} = \text{arg0} - 1$
Операція: $\text{arg0} \leftarrow \text{arg0} - 1$
Приклад: $\text{DEC } R0 ; R0 = R0 - 1$ (декремент регістра)

Конфігурація та операнди

Крок зворотного лічильника.

Вплив на регістр прапорців (FREG)

Оновлює ZF, OF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Компактні декрементні цикли.

Поля з позначкою TBD визначають розширені параметри специфікації.

PUSH

Збереження слова в стек

10110

Стек

Мнемоніка	PUSH
Opcode	10110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Стек
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Збереження слова в стек.

Архітектурна семантика

Сигнатура: PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0
 Операція: SREG <- SREG - 2, MEM[SREG] <- arg0
 Приклад: PUSH R0 ; Зберегти слово R0 на вершину стека

Конфігурація та операнди

Операнд: будь-який регістр R0–R7 або службовий регістр.

Вплив на регістр прапорців (FREG)

Арифметичні прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Апаратна підтримка підпрограм та збереження контексту.

Поля з позначкою TBD визначають розширені параметри специфікації.

POP

10111

Стек

Вилучення слова з вершини стека

Мнемоніка	POP
Opcode	10111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Стек
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Вилучення слова з вершини стека.

Архітектурна семантика

Сигнатура: POP arg0(REG) : arg0 = MEM[SREG], SREG += 2
Операція: arg0 <- MEM[SREG], SREG <- SREG + 2
Приклад: POP R0 ; Витягнути слово з вершини стека в R0

Конфігурація та операнди

Ціль: регістри R0–R7 або регістри стану.

Вплив на регістр прапорців (FREG)

Арифметичні прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Симетричне відновлення контексту переривань.

Поля з позначкою TBD визначають розширені параметри специфікації.

BSL

Барельний зсув ліворуч

11000

Зсуви

Мнемоніка	BSL
Opcode	11000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Зсуви
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Барельний зсув ліворуч.

Архітектурна семантика

Сигнатура: BSL arg0(REG), arg1(REG), arg2(IMM|REG) : arg0 = arg1 « arg2

Операція: arg0 <- arg1 « arg2

FREG.DBIT <- витіснений старший біт (біт 15)

При SMOD=1 у молодший біт подається попередній DBIT; при SMOD=0: 0

Приклад: BSL R0, R1, 1 ; Зсув R1 вліво на 1 біт, результат у R0

Конфігурація та операнди

Величина зсуву: константа або динамічне значення регістра.

Вплив на регістр прапорців (FREG)

Витіснений старший біт фіксується в FREG.DBIT (біт 13). При SMOD=1 у молодший біт подається попередній DBIT; при SMOD=0 заповнюється нулями.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Швидке циклічне та потокове масштабування бітових масивів.

Поля з позначкою TBD визначають розширені параметри специфікації.

BSR

Барельний зсув праворуч

11001

Зсуви

Мнемоніка	BSR
Opcode	11001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Зсуви
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Барельний зсув праворуч.

Архітектурна семантика

Сигнатура: BSR arg0(REG), arg1(REG), arg2(IMM|REG) : arg0 = arg1 » arg2
 Операція: arg0 <- arg1 » arg2
 FREG.DBIT <- витіснений молодший біт (біт 0)
 При SMOD=1 у старший біт подається попередній DBIT; при SMOD=0: 0
 Приклад: BSR R0, R1, 1 ; Зсув R1 вправо на 1 біт, результат у R0

Конфігурація та операнди

Логічний або арифметичний зсув за полем cfg.

Вплив на регістр прапорців (FREG)

Витіснений молодший біт заноситься в DBIT (біт 13). При SMOD=1 у старший біт подається попередній DBIT; при SMOD=0 заповнюється нулями.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Потокова фільтрація та ділення на степені двійки.

Поля з позначкою TBD визначають розширені параметри специфікації.

MSP

11010

Система

Маніпуляція спеціальними регістрами (FREG, PTREG)

Мнемоніка	MSP
Opcode	11010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Система
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Маніпуляція спеціальними регістрами (FREG, PTREG).

Архітектурна семантика

Сигнатура: MSP sreg, [op], args : робота зі спец-регістрами

Операція: **Пряме завантаження (біт 10 = 0):**

sreg=00 (FREG): FREG <- Rs[15:0] (октет 1: [7:5]=Rs)

sreg=01 (PTREG): PTREG <- (Rs_hi << 16) | Rs_lo (октет 1: [7:5]=Rs_hi, [4:2]=Rs_lo)

Модифікація (біт 10 = 1):

sreg=00 (FREG): FREG <- FREG $\odot$ Rs (октет 1: [7:6]=op {AND,OR,XOR,XNOR}, [5:3]=Rs)

sreg=01 (PTREG): PTREG <- PTREG $\pm$ Rs (октет 1: [7]=op {ADD,SUB}, [6:4]=Rs)

Приклад: MSP FREG, R0 ; Прямий запис у FREG значення R0

MSP PTREG, R0, R1 ; Завантаження 24-бітного PTREG = {R0, R1}

MSP FREG, OR, R2 ; Накладання бітової маски: FREG = FREG | R2

MSP PTREG, ADD, R3 ; Зсув покажчика: PTREG = PTREG + R3

Конфігурація та операнди

Регістри FREG (00) та 24-бітний PTREG (01). Підтримує прямий запис, логічні маски FREG і локальне зміщення PTREG.

Вплив на регістр прапорців (FREG)

Прапорці FREG модифікуються напряду або маскою. Арифметика PTREG формує прапорець переносу CF через АЛП.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Універсальний керівний примітив системного рівня.

RSV27

Апаратне розширення

11011

Резерв

Мнемоніка	RSV27
Opcode	11011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Резерв
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратне розширення.

Архітектурна семантика

Сигнатура: RSV27 : зарезервовано
Операція: Reserved for LIM architecture hardware extensions

Конфігурація та операнди

Зарезервовано для майбутніх версій архітектури.

Вплив на регістр прапорців (FREG)

Не визначено.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

RSV28

Апаратне розширення

11100

Резерв

Мнемоніка	RSV28
Opcode	11100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Резерв
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратне розширення.

Архітектурна семантика

Сигнатура: RSV28 : зарезервовано
Операція: Reserved for LIM architecture hardware extensions

Конфігурація та операнди

Зарезервовано для майбутніх версій архітектури.

Вплив на регістр прапорців (FREG)

Не визначено.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

RSV29

Апаратне розширення

11101

Резерв

Мнемоніка	RSV29
Opcode	11101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Резерв
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратне розширення.

Архітектурна семантика

Сигнатура: RSV29 : зарезервовано
Операція: Reserved for LIM architecture hardware extensions

Конфігурація та операнди

Зарезервовано для майбутніх версій архітектури.

Вплив на регістр прапорців (FREG)

Не визначено.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

RSV30

Апаратне розширення

11110

Резерв

Мнемоніка	RSV30
Opcode	11110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Резерв
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратне розширення.

Архітектурна семантика

Сигнатура: RSV30 : зарезервовано
Операція: Reserved for LIM architecture hardware extensions

Конфігурація та операнди

Зарезервовано для майбутніх версій архітектури.

Вплив на регістр прапорців (FREG)

Не визначено.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

RSV31

Апаратне розширення

11111

Резерв

Мнемоніка	RSV31
Opcode	11111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Резерв
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратне розширення.

Архітектурна семантика

Сигнатура: RSV31 : зарезервовано
Операція: Reserved for LIM architecture hardware extensions

Конфігурація та операнди

Зарезервовано для майбутніх версій архітектури.

Вплив на регістр прапорців (FREG)

Не визначено.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

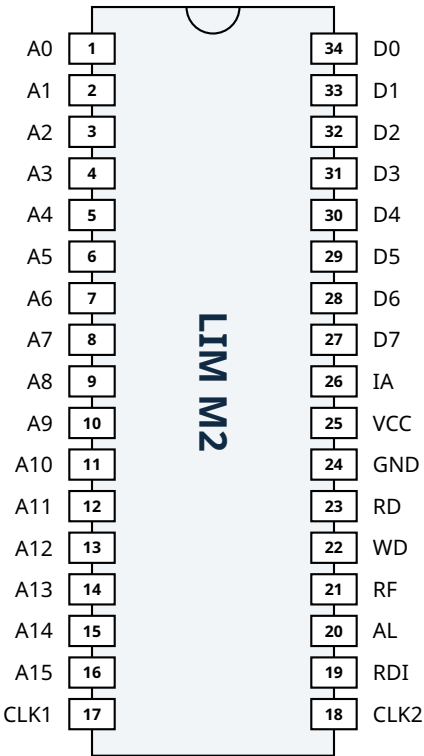
Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

7 Фізичний інтерфейс та цоколівка (34-вивідний корпус)

Процесор реалізовано у надійному компактному 34-вивідному корпусі з дворядним розташуванням:



7.1 Опис системних сигналів

Вивід	Тип	Опис системного сигналу
A0–A15	Вихід / Tri-State	16-розрядна адреса (мультиплексується зовнішнім стробом AL).
D0–D7	Двоспрямований	8-розрядна зовнішня шина обміну з пам'яттю та периферією.
CLK1, CLK2	Вхід	Двофазна тактова синхронізація без перекриття фаз.
VCC, GND	Живлення	Лінії живлення логіки процесора (+5 В номінально).
RD, WD	Вихід	Строби зчитування (Read) та запису (Write) шини.
AL	Вихід	Address Latch: строб фіксації адреси в зовнішньому регістрі.
RF	Вхід	Ready Flag: апаратне підтвердження готовності веденого пристрою.
RDI	Вхід	Raw Direct Interrupt: пріоритетний вхід апаратного переривання.
IA	Вихід	Interrupt Accepted / Interrupt Acknowledge: строб підтвердження прийому переривання; процесор переходить до зчитування номера вектора з шини даних.

8 Підсистема переривань та векторна обробка

Підсистема переривань процесора LIM M2 спроектована для забезпечення мінімальної латентності реакції на події реального часу, строгого детермінізму та повного захисту від шинних конфліктів. Архітектура об'єднує апаратні зовнішні запити та програмні виключення в єдину 16-векторну ієрархію з надійним апаратним квітуванням на фізичному рівні.

8.1 Єдиний простір 16 векторів переривань

Процесор підтримує рівно 16 системних векторів (номери 0–15), які є ортогональними та спільними як для зовнішньої периферії, так і для внутрішніх подій процесорного ядра:

Table 3: Єдина таблиця 16 векторів переривань LIM M2

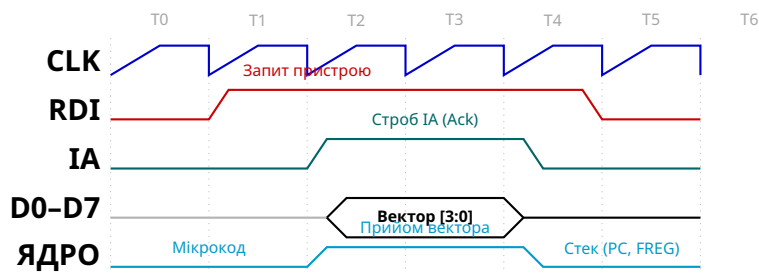
Вектор	Код	Джерело / Тип	Функціональне призначення
VEC 0	0x00	Апаратний / Скидання	Power-on Reset: початкова ініціалізація процесора та регістрів.
VEC 1	0x01	Апаратний (NMI)	Немасковане переривання: збій живлення, критична апаратна аварія.
VEC 2	0x02	Програмна пастка	Арифметичне виключення (ділення на нуль, прапорець FREG. EM).
VEC 3	0x03	Програмна пастка	Системний виклик ядра (System Call / Trap диспетчера ОС).
VEC 4	0x04	Апаратний / Периферія	Системний таймер інтервалів / Квантувач планувальника.
VEC 5	0x05	Апаратний / Периферія	Контролер послідовного інтерфейсу (UART RX/TX).
VEC 6	0x06	Апаратний / Периферія	Контролер прямого доступу до пам'яті (DMA Ready).
VEC 7	0x07	Апаратний / Периферія	Зовнішній контролер переривань / Шинний міст.
VEC 8–15	0x08–0x0F	Спільні (HW / SW)	Користувацькі апаратні лінії та програмні вектори.

8.2 Апаратний протокол квітування (RDI → IA → D0–D7)

Для надійної взаємодії із зовнішніми контролерами реалізовано 4-фазний цикл апаратного рукоштовування (handshake), що повністю виключає необхідність постійного опитування регістрів (polling):

- Фаза запиту (RDI):** Периферійний пристрій встановлює активний високий рівень на вході RDI (*Raw Direct Interrupt*). Лінія опитується ядром на атомарних межах мікрокоманд.
- Фаза перевірки маски (Арбітраж):** Якщо глобальне маскування переривань дозволено (біт FREG. MIE = 1), процесор фіксує запит і завершує поточну неподільну мікрооперацію тракту даних.
- Фаза підтвердження (Строб IA):** Ядро генерує активний строб на виході IA (*Interrupt Accepted / Interrupt Acknowledge*, вивід 26). Цей строб сповіщає всі зовнішні пристрої на шині про готовність ядра прийняти номер вектора.
- Фаза передачі вектора (Шина D0–D7):** Одночасно з видачею IA зовнішня шина даних D0–D7 перемикається ядром у режим приймання (вхід), а лінія зчитування RD активується. Пристрій, що надіслав запит, утримує на лініях D0–D7 8-бітний ідентифікатор вектора (молодші 4 біти адресують вектори 0–15).

5. **Фаза фіксації та перехід:** Процесор фіксує дані з шини D0–D7, знімає сигнал IA та переходить до процедури збереження контексту.



8.3 Збереження контексту у 24-бітному стеку та вхід в обробник

Прийняття переривання дозволено виключно за умови, що процесор не перебуває в стані обробки іншого переривання ($FREG.INTR = 0$) і переривання глобально дозволені ($FREG.MIE = 1$). Якщо $FREG.INTR = 1$, будь-які повторні апаратні переривання апаратно заблоковані.

Під час фіксації переривання ядро виконує таку послідовність захисту стану:

1. **Збереження лічильника команд (PC):** Поточне значення PC зберігається на вершині системного стека за 24-бітним вказівником SREG:

$$SREG \leftarrow SREG - 2, \quad \text{Memory}[SREG] \leftarrow PC_{15:0}$$

2. **Збереження прапорців та статусу (FREG):** Поточний стан регістра FREG (з початковим значенням $FREG.INTR=0$) розміщується у стеку:

$$SREG \leftarrow SREG - 2, \quad \text{Memory}[SREG] \leftarrow FREG$$

3. **Апаратне блокування вкладених переривань:** Ядро виставляє прапорець активності переривання $FREG.INTR \leftarrow 1$ (біт 15) та скидає маску $FREG.MIE \leftarrow 0$, унеможливаючи каскадні повторні входи.
4. **Перехід за вектором:** Адреса обробника обчислюється на основі прийнятого номера вектора та завантажується в PC.

8.4 Повернення з переривання

Вихід з процедури обробки переривання здійснюється відновленням збереженого контексту зі стека:

1. Зі стека зчитується та відновлюється початковий стан регістра статусу:

$$FREG \leftarrow \text{Memory}[SREG], \quad SREG \leftarrow SREG + 2$$

(При цьому біт $FREG.INTR$ автоматично повертається в 0, знімаючи блокування, а біт $FREG.MIE$ відновлює свій попередній стан).

2. Зі стека відновлюється лічильник команд:

$$PC \leftarrow \text{Memory}[SREG], \quad SREG \leftarrow SREG + 2$$

3. Процесор продовжує виконання перерваного потоку інструкцій без жодних втрат даних.

9 Висновки

Архітектура **LIM M2** гармонійно поєднує перевірену надійність мікропрограмного керування та сучасну ефективність активного мультиплексорного тракту без третього стану. Завдяки усуненню шинних колізій та ємнісного розмивання фронтів процесор забезпечує виняткову цілісність сигналів і відкриває безпрецедентний частотний діапазон від 200 МГц до фізичних меж сучасного кремнію.